The Commit Is the Label: Four Problems in Agent Memory That Version Control Already Solves

Frank Guo[1]

[1]Rekal — rekal.dev · guocongmit@gmail.com · github.com/rekal-dev/rekal-cli

DRAFT v0.2 — architecture, benchmark design, and predictions (§6) are final; all empirical values marked ⟨·⟩ are pending the corpus run of §5.

Abstract. Memory systems for AI coding agents are rebuilding, in software, guarantees that version control already provides. An agent's memory is context assembly under a token budget — for each question, the few thousand recorded tokens that change the answer — and the literature assembles it with machinery: tiered stores, memory graphs, and compiled wikis, guarded by model-judged admission and evaluated on hand-annotated benchmarks. The machinery imports four problems it then cannot discharge: **annotation** (no ground truth without human labels), **staleness** (compiled structure must be maintained), **self-confirmation** (a judge that can admit its own mistakes), and **contamination** (one shared pool, one open write path). In the software-engineering setting, each is answered by a git primitive every team already runs: the commit labels which sessions produced which verified change; rebuild and diff make derived structure disposable and its drift reviewable; the merge externally verifies what enters shared memory; and code review is the sole, audited channel across scope boundaries. We build **Rekal**, a local-first, single-binary memory engine, on these four guarantees, and derive **RekalBench**, the first benchmark for repo-grounded intent recall, whose ground truth is mined — not annotated — from the corpus's own commit–session links. On a real working corpus of ⟨*N*⟩ sessions and ⟨*N*⟩ turns, hybrid recall attains ⟨*MRR*⟩ versus ⟨*MRR*⟩ for the strongest baseline (direct grep over raw transcripts), reaches correct answers at ⟨*k*⟩× fewer context tokens, and holds flat with corpus scale where unbounded grep degrades. All predictions were registered before the run (§6); the harness is public, fully local, and runnable by anyone on their own history.

1 Introduction

Code has a ledger; intent does not. Version control records every line a team ships, but the reasoning that produced those lines — explored designs, rejected alternatives, the correction a reviewer shouted mid-session — lives in AI-assistant transcripts that expire with the terminal window. The cost compounds as agents do more of the work: an agent that cannot remember what its own team already tried will confidently re-propose it.

Start from what memory **is** for an agent. The context window is the scarce resource, so memory is not a store — it is **context assembly under a token budget**: for each question, deliver the few thousand tokens that change the answer, out of millions of recorded ones. That framing fixes the requirements. Recall must be **bounded** — a budget-shaped set the agent can drill, not an unbounded scan [1]. What it returns must be **fresh** — reflecting the repository as of the last commit, not a compaction of last month. Every memory must be **lineaged** — dereferenceable to the source turn and the commit it produced, because an unattributable memory cannot be trusted or audited. Anything **shared** must be verified by something other than the model's own judgment. And the whole system must be **cheaply evaluable** — a memory whose quality cannot be measured without an annotation team will not be measured at all.

The research community meets these requirements with **machinery**: organize dialogue history into tiered stores [2], associative graphs [3], or compiled wikis [4]; guard the store with model-judged admission [5]; evaluate on hand-annotated conversational suites [6]. Each piece of machinery then becomes its own research problem, and we name the four: **staleness** — the compiled structure must be maintained; **self-confirmation** — the judge can admit its own mistakes; **annotation** — the evaluation does not scale; **contamination** — the shared pool becomes an attack surface [7]. A recent systematic evaluation from the data-management perspective — twelve memory systems, five workloads, eleven datasets — reaches the fitting verdict: **no single architecture dominates; effectiveness depends on aligning memory structure with the workload** [8].

This paper takes that verdict literally. For coding agents, the workload already has a structure — the repository — and **the four open problems the machinery addresses are already solved by infrastructure every software team runs** (Table 1). The contribution is not a new structure beside git but a memory system built **into** it — plus the benchmark that this construction makes possible, because a corpus whose labels are commits needs no annotators.

Contributions. (1) The design and implementation of Rekal, a local-first, git-native memory engine for coding agents (single binary; capture, storage, hybrid recall, and

Open problem	The literature's machinery	The git-native answer (Rekal mechanism)
Annotation. Where does supervision come from?	Human annotation of dialogue corpora [6]; LLM judges grading themselves	**The commit.** A post-commit hook links sessions to the change they produced — free, objective, abundant (§4.1); RekalBench mines its gold from these links (§5)
Staleness. How does derived memory stay current?	Consolidation daemons; compiled wikis and graphs whose maintenance is "future work" [2], [4]	**Rebuild + diff.** Indexes are disposable and rebuilt (§4.2); materialized views regenerate deterministically, so drift arrives as a reviewable `git diff`
Self-confirmation. Who verifies what enters shared memory?	Self- or consensus-judged admission [5]	**The merge.** Only sessions whose commit landed on the default branch are shared; review + CI + merge are the external verifier (§4.3)
Contamination. How does private memory cross a boundary?	One machine-wide pool, implicitly readable and writable everywhere [2], [9] — the documented contamination surface [7], [10]	**The review.** Cross-repo memory is index-only and structurally unpushable; its sole egress is an origin-labeled page on a PR (§4.4)

Table 1: The thesis. Four problems the agent-memory literature treats as open research, and the git primitive that answers each. The paper walks this table: §4 gives the mechanisms, §6 registers the predictions, §7 reports the measurements.

team sync with no server, API, or telemetry), with an agent-facing skill layer that operationalizes active, multi-step memory reconstruction [3]. (2) **RekalBench**, the first benchmark for repo-grounded intent recall, whose ground truth is mined from the corpus's own commit–session structure rather than annotated. (3) A pre-registered empirical study (predictions fixed before the run, §6) on a real working corpus against the baselines that matter in practice — including the strongest current objection, direct shell interaction over raw transcripts [11] — with token-cost, drill-strategy, and corpus-scale analyses. (4) Ablations isolating each recall signal, enabled by query-time weighting that requires no reindexing.

2 A Worked Example

Everything in this paper appears once, concretely, in the following five-minute story. (Illustrative; the exact JSON shapes are the shipped ones.)

On Monday, an engineer and an agent rework webhook delivery. Mid-session the engineer interrupts: **"don't retry on a fixed delay — it stampedes the downstream on recovery."** The agent switches to exponential backoff with jitter; the change merges. The post-commit hook captures the session — turns, tool calls, the interruption tagged with its own role (`human_steering`) — scrubs secrets, and appends it to the repo's ledger with a link to the commit SHA. Nobody wrote documentation.

On Friday, a different agent, different machine, same team, picks up a related task and asks:

```
$ rekal "should webhook retries use a fixed delay?"
{ "results": [{
    "session_id": "01JNQX8F2K9M",
    "score": 0.87,
    "snippet": "don't retry on a fixed delay - it
       stampedes the downstream on recovery",
    "snippet_turn_index": 41,
    "snippet_role": "human_steering",
    "summary_turn_index": 118,
    "session": { "commit": "9c2f41a",
      "files": ["delivery/retry.go", ...] }
}]}
```

Four properties of the answer carry the paper's argument. The top snippet is the **steering turn itself** — ranking boosts the moments a human redirected the agent, because that is where decisions actually get made. `snippet_role` says a human said this; machine text can never masquerade as intent. The result is a **pointer structure**, not a payload: `snippet_turn_index` locates the evidence, `summary_turn_index` points at a harness-written distillation of the whole session (10–17KB, already paid for during context compaction — never inlined), and the agent drills exactly as deep as the question warrants. And the `commit` field is the provenance anchor: the claim "we ruled this out" dereferences to a change that survived review and merge. The dead end was re-proposed on Friday and ruled out in one query — by a colleague's Monday, not by documentation.

The rest of the paper is this example generalized: §3 the engine that produced it, §4 the four guarantees behind its fields, §5 how a corpus of such sessions labels itself, §6–7 the measurements.

3 The Engine

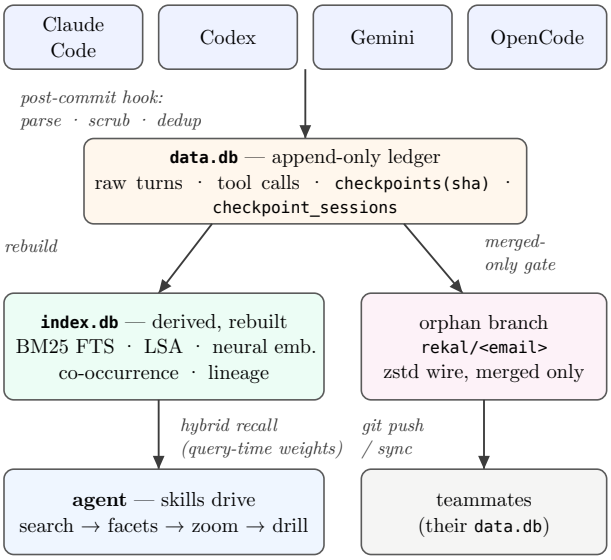

Figure 1: Rekal architecture. One source of truth (append-only `data.db`); all derived structure disposable (`index.db`); sharing gated on merge verification. No server, no API, no telemetry — git is the only wire.

Rekal is one binary embedding its database engine, embedding models, and compression dictionary. A post-commit hook parses the active assistant session(s) — adapters cover Claude Code, Codex, Gemini, and OpenCode — deduplicates turns, scrubs secrets and anonymizes paths **before any byte is stored** (the ordering shown to be the only safe one [10]), and appends to `data.db`. Nothing is summarized at write time: following the preservation result of [12], the unit of storage is the raw turn with role, order, timestamp, and tool-call structure, and every recall result carries source-turn attribution. Write-time compression is lossy and unfixable; query-time distillation improves with every model generation.

One class of distillation is nonetheless **harvested**: when the harness compacts a long session it writes an LLM summary of the conversation so far back into the transcript. Rekal captures these as turns with a dedicated role `summary` — already paid for, cumulative (the latest subsumes the rest), and never confused with human intent. Rekal generates no summaries of its own: a commit-time summary would be query-blind, written before any question exists, whereas the drill is query-aware and paid lazily.

Recall scores sessions by a weighted hybrid of BM25, latent-semantic, and neural similarity; boosts `human_steering` turns (the example's Monday interruption) and, by a smaller factor, harvested summaries; down-weights subagent transcripts; and returns the pointer-structured JSON of §2. Weights live in local config and apply at **query time** — no reindex, which is also what makes the single-signal ablations of §7 free. In RISE's terms [1], search constructs a **bounded interaction space** and the drill tools explore it; a layer of six shipped skills (base search, provenance, self-reflection, distillation, exhaustive census, wiki materialization) is the active-reconstruction policy [3] that sequences those primitives.

4 Four Guarantees from Git

The engine is ordinary; the guarantees are not. This section walks Table 1 row by row, confronting in each row the machinery it replaces.

4.1 Annotation: the commit is the label

Every checkpoint records which sessions produced which commit (`checkpoint_sessions`). This single join is the paper's namesake and its most consequential design choice: it connects a unit of **intent** (the session) to a verified unit of **code** (the commit) — automatically, at zero marginal cost, thousands of times per repository-year. Dialogue-memory research must manufacture this connection by human annotation [6] or accept LLM judges grading their own homework; here the connection is a side effect of using version control. Everything downstream leans on it: provenance answers in recall (§2), the merged-only sharing gate (§4.3), and the entire benchmark (§5), whose five task families are all mined from this one link plus git topology.

4.2 Staleness: rebuild and diff

`data.db` is the only source of truth and is append-only. `index.db` holds everything derived — full-text, embeddings, co-occurrence, lineage, facets — and is **disposable by construction**: any migration, corruption, or model upgrade is handled by deletion and rebuild (content-hash-keyed embedding caching keeps rebuilds incremental). This is the structural answer to the maintenance problem that compiled-memory systems name and defer [2], [4], [13]: **the freshness of a structure you can throw away is not a research problem.**

The same discipline extends to memory the team can browse. A wiki skill materializes topic pages (`docs/wiki/<topic>.md` plus a graph-mapped index) from file co-occurrence clusters and the sessions behind them, shipped as an ordinary pull request. Generation is **deterministic** — topics and edges sorted, thresholds fixed — so a re-run over unchanged history is an empty diff, and any non-empty diff **is** structural drift: an edge appearing in the topic graph is a correlation that did not exist last run; a removed edge is one that decayed; each transition is reviewed before admission. Mutating graph stores cannot show this drift to anyone; here `git log` over the index is a reviewed time series of the project's conceptual structure. The maintenance problem is not solved — it is **converted into code review**, a process teams already run. Because pages are a cache of memory, not the memory, their value is an empirical question: §6 registers the prediction and §7.4 prices the cache (generation cost from the ledger's own lineage records, payoff against recall+drill, decay as a measured rate — the static-notes failure mode made continuous).

4.3 Self-confirmation: the merge is the gate

`rekal push` exports to a per-author git orphan branch only those checkpoints whose commit is reachable from the default branch (with patch-equivalence detection for squash merges); everything else waits, releasing automatically if its branch later lands. The shared tier therefore admits only experience that survived review, CI, and merge — an **external** verification gate, in contrast to the self- or consensus-judged admission proposed to escape the self-confirmation trap [5]: when the same model family executes, summarizes, and admits its own memories, wrong-but-self-consistent trajectories are stored as successes and amplified on reuse. The merge gate has a second, subtler yield: unmerged and abandoned branches remain in the local ledger as labeled **negative** knowledge — the map of dead ends — and §5′s task T3 tests exactly whether a system can warn "we tried this; it didn't land."

4.4 Contamination: review as the egress channel

Rekal's memory has three scopes with asymmetric permeability. The **repo ledger** is truth. The **machine-wide index** optionally folds in the operator's other repos' sessions (explicit opt-in) — index-only, origin-labeled, and structurally unpushable: an imported session has no checkpoint in this repo's ledger, so no code path can export it. The **team wire** admits merged work only. Knowledge crosses a scope boundary exclusively through a human-visible artifact: sessions reach the team when their commit merges, and cross-repo experience becomes committed text through exactly one channel — a wiki page generated in an explicit cross-repo mode that labels every foreign citation with its origin, shipped as a PR whose body declares what is crossing. Machine-wide stores in the literature make the opposite choice [2], [9]: one pool, implicitly readable and writable everywhere — precisely the open write path the security analyses identify as the contamination and exfiltration surface [7], [10], [14], and controlled experiments show sanitization only works **before** summarization, which is why Rekal scrubs before insertion and stores raw. Governance concerns for enterprise memory [15] map onto git's existing controls. The rule compresses to: **wide reads locally, narrow writes globally — egress is always a diff someone approved.**

5 RekalBench

No benchmark exists for repo-grounded intent recall: conversational-memory suites (LoCoMo [6], LongMemEval, PERSONAMEM) evaluate chat-persona recall, and IR suites (BEIR, BRIGHT) evaluate document QA. RekalBench's defining property follows from §4.1: **the corpus labels itself.**

Watch one label being born. The Monday session of §2 checkpointed against commit `9c2f41a`. The miner (plain SQL over the ledger, plus git) emits, with no human in the loop:

```
{ "task": "t1",
  "gold": ["01JNQX8F2K9M"],
  "commit": "9c2f41a",
  "source": { "subject": "webhooks: switch retries
     to exponential backoff",
     "files": ["delivery/retry.go", ...] } }
```

An LLM paraphrases the source material into a natural developer question ("how did we end up handling webhook retry timing?"), a 4-gram Jaccard ceiling (≤ 0.30, one aggressive-paraphrase retry, else the label is dropped and logged) breaks lexical leakage between query and target, and the pair joins the query set. The gold is objective — that session verifiably produced that commit — and the supervision cost is zero. The same construction yields all five task families (Table 2); the steering turn of §2 is likewise a T2 gold, and the never-merged branches of §4.3 are T3 gold **because git says so**.

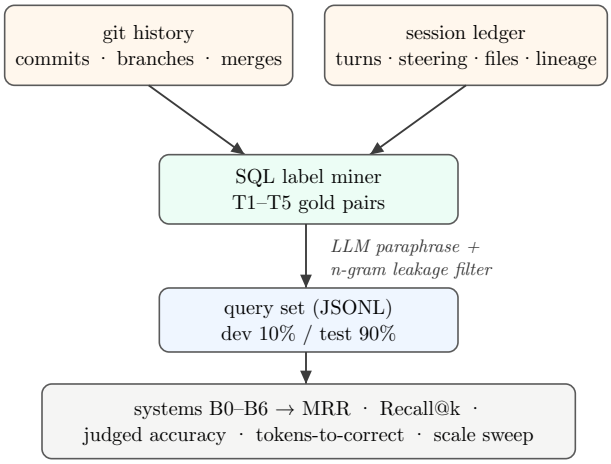

Figure 2: RekalBench pipeline. Labels are mined from structure the tool already records; paraphrase plus an n-gram overlap ceiling breaks lexical leakage between query and target.

5.1 Tasks

Task	Gold label source	Query generation	n
T1 Provenance	commit → producing session(s), via the `checkpoint_sessions` links	paraphrase of commit message + changed paths (not indexed content)	*⟨≈500⟩*
T2 Decision recall	`human_steering` turns	paraphrase of surrounding context; steering turn held out of prompt; 4-gram Jaccard ≤ 0.3	*⟨≈300⟩*
T3 Dead-end awareness	sessions on never-merged branches	"have we tried X?" from the branch's cumulative intent	*⟨≈50⟩*
T4 Multi-hop synthesis	session pairs linked by file co-occurrence / lineage	generator-validated two-session questions	*⟨≈100⟩*
T5 Decision drift	later session reversing an earlier decision on the same files (hand-confirmed sample)	"what is our current approach to X?"	*⟨≈30⟩*

Table 2: RekalBench task families. All labels derive from recorded structure; only T5 needs light manual confirmation. T5 is motivated by the belief-revision result of [16]: memory must surface the **current** decision.

Label noise. A commit's linked sessions can include incidental chatter; we hand-audit 50 T1 pairs and report label precision (P8). **Splits.** 10% dev (tuning allowed) / 90% test, one-shot — the incumbent-versus-candidate discipline of [17].

5.2 Corpus and systems

One operator's real working store, folded per-repo (labels require the repo's own checkpoint ledger) plus machine-wide for scale sweeps. The primary repo alone holds approximately 500 sessions, 63k turns, and 48k tool calls (final corpus card: *⟨repos, sessions, turns, tool calls, steering turns, linked commits, date range⟩*). No session content leaves the machine; the paper reports aggregates only.

ID	System
B0	No memory — question (plus repo code where the task allows) only
B1	Grep/DCI [11] — same agent model with `rg`/`jq`/`sed` over raw transcript JSONL; GrepSeek-informed prompt [18]; equal turn budget
B2	Static notes — one-time LLM-distilled `MEMORY.md` (≤8k tokens), the folk practice
B3	BM25-only (Rekal weights `{1,0,0}`) — query-time ablation, no reindex
B4	Neural-only (weights `{0,0,1}`) — ablation
B5	Rekal full hybrid + steering boost + summary boost + subagent down-weight
B6	Rekal + skills — B5 driven by the shipped playbooks (rungs 2–4)

Table 3: Systems under test. B1 is not a straw man: direct corpus interaction beats sparse, dense, and reranking retrieval on several published benchmarks [11], [18] — it is the strongest current objection to any index.

Metrics. Rung 1 (retrievability): MRR, Recall@{1, 5, 10}, nDCG@10 with bootstrap CIs. Rung 2 (answer quality): LLM-judged correctness against the gold turn (distinct generate/answer/judge models; 50-sample human agreement check). Rung 3 (efficiency): context tokens to first correct answer, wall-clock, dollar cost — the axis RISE and MRAgent make primary [1], [3] — plus a **judge-free drill-strategy proxy**: on queries where recall reaches the gold session, the raw window around the matched turn versus the single turn `summary_turn_index` points at, compared on tokens ingested and gold-term coverage. It measures context-assembly cost, not answer quality, and runs with no LLM in the loop. The **L3 gate** (wiki experiment) prices the browsing cache of §4.2: generation cost from the ledger's own workflow lineage, payoff against recall+drill on broad queries, decay as regeneration diff rate, and a cross-repo A/B (own-repo evidence versus reviewed-egress cross-repo mode). Scale sweep: metrics and B1 latency at 10/25/50/100% date-cut subsets. Freshness: recall bucketed by target-session age; rebuild wall-clock versus corpus size. Rung 4 (agent-in-the-loop): 10–20 real tasks, A/B on steering count, dead-end re-proposals, time-to-done.

6 Predictions (registered before the run)

The four guarantees of Table 1 divide by **how they are verified**. Self-confirmation and contamination are answered by construction: the merge gate and the egress restriction are code paths, checkable by reading them — an imported session has no checkpoint, so no export path exists; no experiment can add to that. What experiments test is each guarantee's **yield**: whether the commit-link supervision produces clean labels (P8 — annotation), whether the freshness discipline is affordable and the browsing cache earns its keep (P6, P7 — staleness), whether the negative knowledge the merge gate preserves is actually recallable (T3 inside P1 — self-confirmation), and whether reviewed

cross-repo egress buys measurable coverage (P7′s A/B — contamination). P1–P5 are the table stakes beneath all four: a memory whose recall loses to grep needs no philosophy.

Stated falsifiably, before any table is filled; §7′s tables are keyed to them. Where a prediction fails, the failure is reported, not reframed.

1. **P1 (retrievability).** B5 beats B1 on pooled test-split MRR and nDCG@10 with non-overlapping bootstrap CIs.
2. **P2 (signals).** B5 beats each single-signal ablation (B3, B4) pooled; disabling the steering boost measurably hurts T2.
3. **P3 (drill strategies).** The trade is question-shaped: summary-first wins gold-term coverage on broad tasks (T1/T3); the raw window wins coverage-per-token on pointed lookups (T2).
4. **P4 (judged efficiency).** B5/B6 match or beat B1′s judged accuracy at ≥ 2× fewer tokens-to-correct.
5. **P5 (scale, the RISE crossover).** B1′s accuracy and latency degrade monotonically with corpus size; B5 holds flat within CI. A crossover point exists on session data [1].
6. **P6 (freshness — staleness).** Rung-1 quality shows no decay with target-session age; full index rebuild stays under minutes at the corpus's full size.
7. **P7 (the L3 gate — staleness and contamination).** Wiki pages beat recall+drill on tokens for broad queries at comparable coverage; pages decay at a measurable weekly rate; cross-repo mode adds coverage on topics that span repos. If pages age fast or lose on coverage, the cached L3 layer is not built — that negative result is a finding of this paper, not a gap in it.
8. **P8 (label validity — annotation).** T1 label precision ≥ 0.9 on a 50-pair human audit.
9. **P9 (embedding substitution).** *Registered after the first rung-1 run motivated it, before any embedding measurement* — the one prediction in this list with that provenance, stated so the reader can weigh it. The motivating finding: the neural-only ablation (B4) was the weakest signal and dev tuning pushed its layer weight down. The prediction: substituting a stronger code-tuned embedding model for the embedded one (a config-only swap to the OpenAI-compatible HTTP backend; the content-hash-keyed cache makes the swap and the swap back one reindex each) lifts neural-only (B4′ over B4) materially, but lifts the hybrid (B5′ over B5) by little — on a single-repo corpus the question and its answering session share vocabulary (identifiers, error strings, file names), so the binding constraint is workload alignment, not embedding quality. Either direction is a finding: a large hybrid lift changes the shipped default model and weights; a null lift is direct evidence for this paper's alignment-by-construction premise [8].

7 Results

All values in this section are placeholders pending the corpus run; tables define the exact shape of the report and are keyed to §6′s predictions.

7.1 Retrievability (P1, P2)

System	T1 MRR	T1 R@5	T2 MRR	T3 R@5	T4 b@10
B1 grep/DCI	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩
B2 static notes	—	—	—	—	—
B3 BM25-only	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩
B4 neural-only	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩
B5 Rekal hybrid	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩

Table 4: Retrieval quality by task (test split, bootstrap 95% CIs). Verdicts: P1 ⟨*holds / fails*⟩, P2 ⟨*holds / fails*⟩. B2 has no per-query retrieval and is evaluated at rungs 2–3 only.

7.2 Embedding substitution (P9)

System	Pooled MRR	T1 MRR	T2 MRR
B4 neural-only, embedded model	⟨·⟩	⟨·⟩	⟨·⟩
B4′ neural-only, substituted	⟨·⟩	⟨·⟩	⟨·⟩
B5 hybrid, embedded model	⟨·⟩	⟨·⟩	⟨·⟩
B5′ hybrid, substituted	⟨·⟩	⟨·⟩	⟨·⟩

Table 5: Embedding-model substitution (test split, same weights within each pairing; substituted model served over the HTTP backend, model id in the run manifest). Verdict: P9 ⟨*holds / fails*⟩.

7.3 Drill strategies (P3)

Drill strategy	Tokens	Coverage	Cov. / 1k tok
window (5-turn, at match)	⟨·⟩	⟨·⟩	⟨·⟩
summary-first (pointer)	⟨·⟩	⟨·⟩	⟨·⟩

Table 6: Judge-free drill-strategy proxy, paired on queries where recall reaches the gold session and a compaction summary exists. Verdict: P3 ⟨*holds / fails*⟩ (per-task split reported alongside).

7.4 Answer quality and token cost (P4)

System	Judged acc.	Tokens → correct	$ / query
B0 no memory	⟨·⟩	—	—
B1 grep/DCI	⟨·⟩	⟨·⟩	⟨·⟩
B2 static notes	⟨·⟩	⟨·⟩	⟨·⟩
B5 Rekal	⟨·⟩	⟨·⟩	⟨·⟩
B6 Rekal + skills	⟨·⟩	⟨·⟩	⟨·⟩

Table 7: Answer quality and efficiency on a 200-query stratified subset. Verdict: P4 ⟨*holds / fails*⟩.

7.5 The L3 gate (P7)

Measure	Value	Source
pages generated / sessions cited per page	⟨ · ⟩	wiki-run PR
generation: turns + tool calls per page	⟨ · ⟩	ledger (`workflow_name` lineage)
generation: tokens ingested per page	⟨ · ⟩	harness accounting
broad-query answering: page vs recall+drill, tokens	⟨ · *vs* · ⟩	proxy, drill-table columns
broad-query answering: page vs recall+drill, coverage	⟨ · *vs* · ⟩	proxy, drill-table columns
regeneration diff rate (pages invalidated / week)	⟨ · ⟩	watermark vs new sessions
cross-repo mode: pages with foreign citations / coverage delta	⟨ · / · ⟩	origin labels; A/B on generation mode

Table 8: The wiki experiment prices the browsing cache: generation cost is measured from the ledger's own lineage records — the memory system accounts for its own distillation. Verdict: P7 *⟨holds / fails / cache not built⟩*.

7.6 Scale and freshness (P5, P6)

We re-run rung 1 at date-cut corpus subsets. Deliverables: accuracy-versus-corpus-size and tokens-versus-accuracy curves (Fig. *⟨3⟩*, *⟨4⟩*); B1 latency and failure rate versus corpus size; rebuild wall-clock versus turns; rung-1 quality bucketed by target-session age. Verdicts: P5 *⟨holds / fails⟩*, P6 *⟨holds / fails⟩*.

7.7 Agent-in-the-loop (rung 4)

⟨10–20 tasks; steering interventions, dead-end re-proposals, time-to-done, with and without Rekal⟩ — run last, only if rungs 1–3 hold.

8 Positioning

The design confrontations live inline in §4; what remains is where Rekal sits in the field's own coordinate systems.

On the four-module anatomy of the data-management evaluation [8] — representation & storage, extraction, retrieval & routing, maintenance — Rekal classifies as a multi-paradigm hybrid on the first three: heterogeneous multi-engine storage (relational + full-text + vector in one embedded database), **deterministic** extraction (parsing and harvesting; uniquely among the taxonomy's systems, no LLM sits anywhere in the write path — the property the contamination analyses require), and multi-stage hybrid retrieval with agentic routing via the skill layer. On the fourth module it exits the taxonomy's design space entirely: that survey's maintenance column enumerates LLM-driven semantic consolidation, capacity-driven eviction, and timestamp multi-versioning — each a paid, lossy, or complexity-bearing liability — whereas Rekal's maintenance is **rebuild, diff, and review**: nothing is consolidated (append-only), nothing is evicted (raw is cheap), and versioning is git itself. The same evaluation's no-free-lunch finding — effectiveness depends on aligning memory structure with the workload — is this paper's premise taken to its limit: the coding workload's structure is the repository, and Rekal aligns by construction rather than by tuning. On the **compression axis** — aggressive consolidation [2] versus preservation with attribution [12] — Rekal takes EMem's side at the storage layer (raw turns, attributed snippets) and harvests, rather than generates, distillation. On the **retrieval axis** — one-shot lookup versus active reconstruction [3] — it takes MRAgent's side at the interaction layer: the skill playbooks drive iterative search–facet–zoom–drill loops over engine primitives, and LLM-Wiki's page-traversal result [4] is recovered, without the compilation liability, as the reviewed wiki of §4.2. Architecturally the storage is aligned with SAG's query-time joins over flat indexes [13]. Against retrieval-free direct corpus interaction [11], [18] the position is conditional and matches RISE [1]: below some corpus size grep is fine — Rekal still adds parsing, scrubbing, provenance, and team sync — and above it a bounded interaction space is the difference between answers and wall-clock failures; the crossover on session data is an output of this work (P5). Self-improvement from traces [9], [17], [19] is the flywheel this corpus natively enables (§9); LoCoMo-style persona memory [6] is out of scope by design.

9 Limitations

Rung 1 is a retrievability proxy — a hit is not a useful answer; that is why rungs 2–4 exist, and why P3/P7 are labeled cost-coverage proxies, never answer quality. Self-labels are noisy; P8 quantifies it and results are reported with label-precision-adjusted upper bounds. A single-operator corpus is a case study until replicated — the honest path to multi-corpus evidence is that the harness is public, fully local, and runnable by any Rekal user on their own store, at zero annotation cost. The wire format carries no harness identity for cross-team sessions today; the worked example's JSON is illustrative pending the corpus run's substitution of a real (redacted) instance.

10 The Flywheel, and What This Buys

The corpus contains its own improvement signal: which recalled sessions an agent drills into after a search is an implicit relevance label [19], enabling private per-corpus weight tuning accepted only on head-to-head wins over the incumbent configuration [17]; trajectory review can revise the skill layer itself [9]; and the wiki's generation runs are themselves ledgered sessions, so even the distillation loop is self-accounting. None of this requires new machinery — it is the same ledger, read again.

Memory for coding agents does not need another compiled structure to maintain or another self-judged store to poison itself. It needs a ledger that already has ground truth. Git supplies the label (the commit), the freshness mecha-

nism (rebuild and diff), the verifier (the merge), and the egress channel (the review); Rekal supplies the capture, the disposable indexes, the bounded recall, and the playbooks. RekalBench turns the same structure into the first benchmark for repo-grounded intent recall — and every prediction in §6 is falsifiable by running it, locally, on your own history.

Reproducibility. Engine, skills, benchmark spec, extraction SQL, runbook (`DATA-RUN.md`), and this paper's source: github.com/rekal-dev/rekal-cli (`docs/research/`). All benchmark data remains on the operator's machine; published artifacts are aggregates, prompts, and code.

References

[1] S. Zhuang and others, "Towards Retrieving Interaction Spaces for Agentic Search." [Online]. Available: https://arxiv.org/abs/2606.06880

[2] S. Yan, J. Ni, L. Zheng, and others, "AdaMem: Adaptive User-Centric Memory for Long-Horizon Dialogue Agents." [Online]. Available: https://arxiv.org/abs/2603.16496

[3] S. Ji, Y. Li, and B. Hooi, "Memory is Reconstructed, Not Retrieved: Graph Memory for LLM Agents." [Online]. Available: https://arxiv.org/abs/2606.06036

[4] H. Ming, F. Li, X. Wu, W. Que, and others, "Retrieval as Reasoning: Self-Evolving Agent-Native Retrieval via LLM-Wiki." [Online]. Available: https://arxiv.org/abs/2605.25480

[5] S. Zhu, Y. Qi, Y. Wang, and others, "Escaping the Self-Confirmation Trap: An Execute-Distill-Verify Paradigm for Agentic Experience Learning." [Online]. Available: https://arxiv.org/abs/2606.24428

[6] A. Maharana and others, "Evaluating Very Long-Term Conversational Memory of LLM Agents." [Online]. Available: https://arxiv.org/abs/2402.17753

[7] Survey authors, "A Survey on Long-Term Memory Security in LLM Agents: Attacks, Defenses, and Governance Across the Memory Lifecycle." [Online]. Available: https://arxiv.org/abs/2604.16548

[8] W. Zhou and others, "Are We Ready For An Agent-Native Memory System?." [Online]. Available: https://arxiv.org/abs/2606.24775

[9] S. Wu, H. Zhu, Y. Zhang, X. Wang, and S. Yeung-Levy, "AutoMem: Automated Learning of Memory as a Cognitive Skill." [Online]. Available: https://arxiv.org/abs/2607.01224

[10] State-contamination authors, "State Contamination in Memory-Augmented LLM Agents." [Online]. Available: https://arxiv.org/abs/2605.16746

[11] Z. Li, H. Zhang, and others, "Beyond Semantic Similarity: Rethinking Retrieval for Agentic Search via Direct Corpus Interaction." [Online]. Available: https://arxiv.org/abs/2605.05242

[12] S. Zhou and J. Han, "A Simple Yet Strong Baseline for Long-Term Conversational Memory of LLM Agents." [Online]. Available: https://arxiv.org/abs/2511.17208

[13] Y. Wu, J. Li, X. Liang, and others, "SAG: SQL-Retrieval Augmented Generation with Query-Time Dynamic Hyperedges." [Online]. Available: https://arxiv.org/abs/2606.15971

[14] Survey authors, "From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents." [Online]. Available: https://arxiv.org/abs/2606.04990

[15] H. Taheri, "Governed Memory: A Production Architecture for Multi-Agent Workflows." [Online]. Available: https://arxiv.org/abs/2603.17787

[16] BeliefShift authors, "BeliefShift: Benchmarking Temporal Belief Consistency and Opinion Drift in LLM Agents." [Online]. Available: https://arxiv.org/abs/2603.23848

[17] W. Pan and others, "Evolving Agents in the Dark: Retrospective Harness Optimization via Self-Preference." [Online]. Available: https://arxiv.org/abs/2606.05922

[18] GrepSeek authors, "Training Search Agents for Direct Corpus Interaction." [Online]. Available: https://arxiv.org/abs/2605.29307

[19] Y. Zhou and others, "Learning to Retrieve from Agent Trajectories." [Online]. Available: https://arxiv.org/abs/2604.04949